

Python语言与系统设计

史骏

juns@hfut.edu.cn

内容

01 从数据处理到人工智能

02 从Web解析到网络空间

03 从人机交互到艺术设计

01 从数据处理到人工智能

从数据处理到人工智能

数据表示->数据清洗->数据统计->数据可视化->数据挖掘->人工智能

- 数据表示：采用合适方式用程序表达数据
- 数据清理：数据归一化、数据转换、异常值处理
- 数据统计：数据的概要理解，数量、分布、中位数等

从数据处理到人工智能

数据表示->数据清洗->数据统计->数据可视化->数据挖掘->人工智能

- 数据可视化：直观展示数据内涵的方式
- 数据挖掘：从数据分析获得知识，产生数据外的价值
- 人工智能：数据/语言/图像/视觉等方面深度分析与决策

从数据处理到人工智能

- Python库之数据分析
- Python库之数据可视化
- Python库之文本处理
- Python库之机器学习

Python库之数据分析



NumPy: 表达N维数组的最基础库

- Python接口使用，C语言实现，计算速度优异
- Python数据分析及科学计算的基础库，支撑Pandas等
- 提供直接的矩阵运算、广播函数、线性代数等功能

http://www.numpy.org

NumPy的引用

```
import numpy as np
```

引入模块的别名

尽管别名可以省略或更改，建议使用上述约定的别名

N维数组对象：ndarray

ndarray是一个多维数组对象，由两部分构成：

- 实际的数据
- 描述这些数据的元数据（数据维度、数据类型等）

ndarray数组一般要求所有元素类型相同（同质），数组下标从0开始

ndarray实例

```
Python 3.6.5 |Anaconda, Inc.| (default, Mar 29 2018, 13:32:41) [MSC v.1900 64 bit
(AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 6.4.0 -- An enhanced Interactive Python.

In [1]: a = np.array([[0, 1, 2, 3, 4],
   ...:                [9, 8, 7, 6, 5]])
   ...:
Traceback (most recent call last):

  File "<ipython-input-1-c928340cbaf3>", line 1, in <module>
    a = np.array([[0, 1, 2, 3, 4],

NameError: name 'np' is not defined
```

ndarray对象的属性

属性	说明
.ndim	维度的数量
.shape	ndarray对象的尺度，对于矩阵，n行m列
.size	ndarray对象元素的个数，相当于.shape中n*m的值
.dtype	ndarray对象的元素类型
.itemsize	ndarray对象中每个元素的大小，以字节为单位

ndarray数组的创建方法

（1）从Python中的列表、元组等类型创建ndarray数组

```
x = np.array(list/tuple)

x = np.array(list/tuple, dtype=np.float32)
```

当np.array()不指定dtype时，NumPy将根据数据情况关联一个dtype类型

ndarray数组的创建方法

（1）从Python中的列表、元组等类型创建ndarray数组

```
In [23]: x = np.array([0, 1, 2, 3])

In [24]: print(x)
[0 1 2 3]

In [25]: x = np.array((4, 5, 6, 7))

In [26]: print(x)
[4 5 6 7]

In [27]: x = np.array([ [1, 2], [9, 8], (0.1, 0.2)])

In [28]: print(x)
[[1.  2. ]
 [9.  8. ]
 [0.1 0.2]]
```

从列表类型创建

从元组类型创建

从列表和元组混合类型创建

ndarray数组的创建方法

（2）使用NumPy中函数创建ndarray数组，如：arange, ones, zeros等

函数	说明
np.arange(n)	类似range()函数，返回ndarray类型，元素从0到n-1
np.ones(shape)	根据shape生成一个全1数组，shape是元组类型
np.zeros(shape)	根据shape生成一个全0数组，shape是元组类型
np.full(shape,val)	根据shape生成一个数组，每个元素值都是val
np.eye(n)	创建一个正方的n*n单位矩阵，对角线为1，其余为0

ndarray数组的创建方法

（2）使用NumPy中函数创建ndarray数组，如：arange, ones, zeros等

函数	说明
np.ones_like(a)	根据数组a的形状生成一个全1数组
np.zeros_like(a)	根据数组a的形状生成一个全0数组
np.full_like(a,val)	根据数组a的形状生成一个数组，每个元素值都是val

ndarray数组的创建方法

（3）使用NumPy中其他函数创建ndarray数组

函数	说明
np.linspace()	根据起止数据等间距地填充数据，形成数组
np.concatenate()	将两个或多个数组合并成一个新的数组

ndarray数组的维度变换

方法	说明
.reshape(shape)	不改变数组元素，返回一个shape形状的数组，原数组不变
.resize(shape)	与.reshape()功能一致，但修改原数组
.swapaxes(ax1,ax2)	将数组n个维度中两个维度进行调换
.flatten()	对数组进行降维，返回折叠后的一维数组，原数组不变

ndarray数组的索引和切片

一维数组的索引和切片：与Python的列表类似

```
In [76]: a = np.array([9, 8, 7, 6, 5])

In [77]: a[2]
Out[77]: 7

In [78]: a[1 : 4 : 2]
Out[78]: array([8, 6])
```

起始编号：终止编号(不含)：步长，3元素冒号分割

编号0开始从左递增，或-1开始从右递减

ndarray数组的索引和切片

多维数组的索引

```
In [79]: a = np.arange(24).reshape((2,3,4))

In [80]: a
Out[80]:
array([[[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]],

       [[12, 13, 14, 15],
        [16, 17, 18, 19],
        [20, 21, 22, 23]]])

In [81]: a[1, 2, 3]
Out[81]: 23

In [82]: a[0, 1, 2]
Out[82]: 6

In [83]: a[-1, -2, -3]
Out[83]: 17
```

NumPy一元函数

对ndarray中的数据执行元素级运算的函数

函数	说明
np.abs(x) np.fabs(x)	计算数组各元素的绝对值
np.sqrt(x)	计算数组各元素的平方根
np.square(x)	计算数组各元素的平方
np.log(x) np.log10(x) np.log2(x)	计算数组各元素的自然对数、10底对数和2底对数
np.ceil(x) np.floor(x)	计算数组各元素的ceiling值 或 floor值

NumPy一元函数

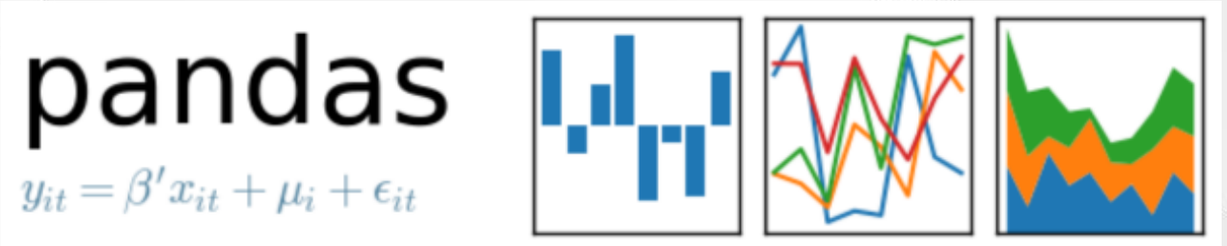
函数	说明
np.rint(x)	计算数组各元素的四舍五入值
np.modf(x)	将数组各元素的小数和整数部分以两个独立数组形式返回
np.cos(x) np.cosh(x) np.sin(x) np.sinh(x) np.tan(x) np.tanh(x)	计算数组各元素的普通型和双曲型三角函数
np.exp(x)	计算数组各元素的指数值
np.sign(x)	计算数组各元素的符号值，1(+)，0，-1(-)

NumPy二元函数

函数	说明
+ - * / **	两个数组各元素进行对应运算
np.maximum(x, y) np.minimum(x, y) np.fmax() np.fmin()	元素级的最大值/最小值计算
np.mod(x,y)	元素级的模运算
np.copysign(x,y)	将数组y中各元素值的符号赋值给数组x对应元素
> < >= <= == !=	算术比较，产生布尔型数组

Python库之数据分析

Pandas: Python数据分析高层次应用库



- 提供了简单易用的数据结构和数据分析工具
- 理解数据类型与索引的关系，操作索引即操作数据
- Python最主要的数据分析功能库，基于Numpy开发

http://pandas.pydata.org

Pandas库的引用

Pandas是Python第三方库，提供高性能易用数据类型和分析工具

```
import pandas as pd
```

Pandas基于NumPy实现，常与NumPy和Matplotlib一同使用

Pandas库的理解

两个数据类型

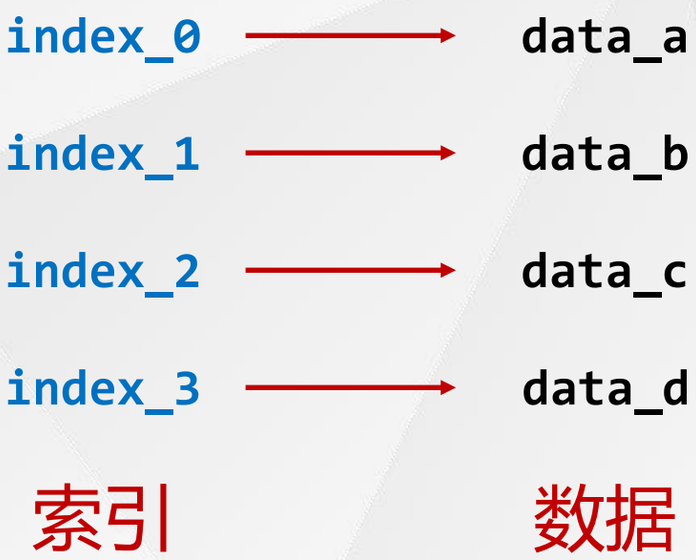
Series = 索引 + 一维数据

DataFrame = 行列索引 + 二维数据

基于上述数据类型的各类操作
基本操作、运算操作、特征类操作、关联类操作

Series类型

Series类型由一组数据及与之相关的数据索引组成



Series类型

Series类型由一组数据及与之相关的数据索引组成

```
In [111]: import pandas as pd
     ...:

In [112]: a = pd.Series([9, 8, 7, 6])

In [113]: a
Out[113]:
0    9
1    8
2    7
3    6
dtype: int64
```

自动索引

NumPy中数据类型

Series类型

Series类型由一组数据及与之相关的数据索引组成

```
In [117]: import pandas as pd

In [118]: b = pd.Series([9, 8, 7, 6], index=['a', 'b', 'c', 'd'])

In [119]: b
Out[119]:
a    9
b    8
c    7
d    6
dtype: int64
```

自定义索引

作为第二个参数，可以省略index=

Series类型的基本操作

Series类型包括index和values两部分

Series类型的操作类似ndarray类型

Series类型的操作类似Python字典类型

Series类型的基本操作

```
In [1]: import pandas as pd

In [2]: aa = pd.Series([9, 8, 7, 6], ['a', 'b', 'c', 'd'])

In [3]: aa
Out[3]:
a    9
b    8
c    7
d    6
dtype: int64

In [4]: aa.index
Out[4]: Index(['a', 'b', 'c', 'd'], dtype='object')

In [5]: aa.values
Out[5]: array([9, 8, 7, 6], dtype=int64)
```

.index 获得索引

.values 获得数据

Series类型的基本操作

Series类型的操作类似ndarray类型：

- 索引方法相同，采用[]
- NumPy中运算和操作可用于Series类型
- 可以通过自定义索引的列表进行切片
- 可以通过自动索引进行切片，如果存在自定义索引，则一同被切片

Series类型的基本操作

```
In [14]: aa = pd.Series([9, 8, 7, 6, 5], ['a', 'b', 'c', 'd', 'e'])

In [15]: aa
Out[15]:
a    9
b    8
c    7
d    6
e    5
dtype: int64

In [16]: aa[3]
Out[16]: 6

In [17]: aa[:3]
Out[17]:
a    9
b    8
c    7
dtype: int64
```

```
In [22]: aa[aa > aa.median()]
Out[22]:
a    9
b    8
dtype: int64

In [23]: np.exp(aa)
Out[23]:
a    8103.083928
b    2980.957987
c    1096.633158
d     403.428793
e     148.413159
dtype: float64
```

Series类型的基本操作

Series类型的操作类似Python字典类型：

- 通过自定义索引访问
- 保留字in操作
- 使用.get()方法

DataFrame类型



DataFrame是一个表格型的数据类型，每列值类型可以不同

DataFrame既有行索引、也有列索引

DataFrame常用于表达二维数据，但可以表达多维数据

DataFrame类型

从二维ndarray对象创建

```
In [61]: d = pd.DataFrame(np.arange(10).reshape(2,5))

In [62]: d
Out[62]:
   0  1  2  3  4
0  0  1  2  3  4
1  5  6  7  8  9
```

自动列索引

自动行索引

DataFrame类型

从一维ndarray对象字典创建

```
In [4]: import pandas as pd

In [5]:  dt = {'one': pd.Series([1, 2, 3], index=['a', 'b', 'c']),
   ...:        'two': pd.Series([9, 8, 7, 6], index=['a', 'b', 'c', 'd'])}
   ...:

In [6]: d = pd.DataFrame(dt)

In [7]: d
Out[7]:
   one  two
a  1.0    9
b  2.0    8
c  3.0    7
d  NaN    6

In [8]: pd.DataFrame(dt, index=['b', 'c', 'd'], columns=['two', 'three'])
Out[8]:
   two three
b    8   NaN
c    7   NaN
d    6   NaN
```

自动列索引

自动行索引

数据根据行列索引自动补齐

DataFrame类型

城市	环比	同比	定基
北京	101.5	120.7	121.4
上海	101.2	127.3	127.8
广州	101.3	119.4	120.0
深圳	102.0	140.0	145.5
沈阳	100.0	101.4	101.6

```
In [12]: dl = {'城市': ['北京', '上海', '广州', '深圳', '沈阳'],
    ...:       '环比': [101.5, 101.2, 101.3, 102.0, 100.1],
    ...:       '同比': [120.7, 127.3, 119.4, 140.9, 101.4],
    ...:       '定基': [121.4, 127.8, 120.0, 145.5, 101.6]}
    ...:

In [13]: d = pd.DataFrame(dl, index=['c1', 'c2', 'c3', 'c4', 'c5'])

In [14]: d
Out[14]:
      同比  城市     定基     环比
c1  120.7  北京  121.4  101.5
c2  127.3  上海  127.8  101.2
c3  119.4  广州  120.0  101.3
c4  140.9  深圳  145.5  102.0
c5  101.4  沈阳  101.6  100.1
```

```
In [15]: d.index
Out[15]: Index(['c1', 'c2', 'c3', 'c4', 'c5'], dtype='object')

In [16]: d.columns
Out[16]: Index(['同比', '城市', '定基', '环比'], dtype='object')

In [17]: d.values
Out[17]:
array([[120.7, '北京', 121.4, 101.5],
       [127.3, '上海', 127.8, 101.2],
       [119.4, '广州', 120.0, 101.3],
       [140.9, '深圳', 145.5, 102.0],
       [101.4, '沈阳', 101.6, 100.1]], dtype=object)
```

重新索引

.reindex(index=None, columns=None, …)的参数

参数	说明
index, columns	新的行列自定义索引
fill_value	重新索引中，用于填充缺失位置的值
method	填充方法, ffill当前值向前填充，bfill向后填充
limit	最大填充量
copy	默认True，生成新的对象，False时，新旧相等不复制

索引类型的常用方法

方法	说明
.append(idx)	连接另一个Index对象，产生新的Index对象
.diff(idx)	计算差集，产生新的Index对象
.intersection(idx)	计算交集
.union(idx)	计算并集
.delete(loc)	删除loc位置处的元素
.insert(loc,e)	在loc位置增加一个元素e

删除指定索引对象

.drop()能够删除Series和DataFrame指定行或列索引

```
In [86]: a = pd.Series([9, 8, 7, 6], index=['a', 'b', 'c', 'd'])

In [87]: a
Out[87]:
a    9
b    8
c    7
d    6
dtype: int64

In [88]: a.drop(['b', 'c'])
Out[88]:
a    9
d    6
dtype: int64
```

```
In [83]: d
Out[83]:
    城市     同比     环比     定基
c5  沈阳  101.4  100.1  101.6
c4  深圳  140.9  102.0  145.5
c3  广州  119.4  101.3  120.0
c2  上海  127.3  101.2  127.8
c1  北京  120.7  101.5  121.4

In [84]: d.drop('c5')
Out[84]:
    城市     同比     环比     定基
c4  深圳  140.9  102.0  145.5
c3  广州  119.4  101.3  120.0
c2  上海  127.3  101.2  127.8
c1  北京  120.7  101.5  121.4

In [85]: d.drop('同比', axis=1)
Out[85]:
    城市     环比     定基
c5  沈阳  100.1  101.6
c4  深圳  102.0  145.5
c3  广州  101.3  120.0
c2  上海  101.2  127.8
c1  北京  101.5  121.4
```

算术运算规则

算术运算根据行列索引，补齐后运算，运算默认产生浮点数

补齐时缺项填充NaN (空值)

二维和一维、一维和零维间为广播运算

采用+ - * /符号进行的二元运算产生新的对象

数据类型的算术运算

```
In [90]: a = pd.DataFrame(np.arange(12).reshape(3,4))

In [91]: a
Out[91]:
   0  1   2   3
0  0  1   2   3
1  4  5   6   7
2  8  9  10  11

In [92]: b = pd.DataFrame(np.arange(20).reshape(4,5))

In [93]: b
Out[93]:
    0   1   2   3   4
0   0   1   2   3   4
1   5   6   7   8   9
2  10  11  12  13  14
3  15  16  17  18  19
```

```
In [94]: a + b
Out[94]:
      0     1     2     3   4
0   0.0   2.0   4.0   6.0 NaN
1   9.0  11.0  13.0  15.0 NaN
2  18.0  20.0  22.0  24.0 NaN
3   NaN   NaN   NaN   NaN NaN

In [95]: a * b
Out[95]:
      0     1      2      3   4
0   0.0   1.0    4.0    9.0 NaN
1  20.0  30.0   42.0   56.0 NaN
2  80.0  99.0  120.0  143.0 NaN
3   NaN   NaN    NaN    NaN NaN
```

自动补齐，缺项补NaN

数据类型的算术运算

```
In [96]: a = pd.DataFrame(np.arange(12).reshape(3,4))

In [97]: a
Out[97]:
   0  1   2   3
0  0  1   2   3
1  4  5   6   7
2  8  9  10  11

In [98]: b = pd.DataFrame(np.arange(20).reshape(4,5))

In [99]: b
Out[99]:
    0   1   2   3   4
0   0   1   2   3   4
1   5   6   7   8   9
2  10  11  12  13  14
3  15  16  17  18  19
```

```
In [100]: b.add(a, fill_value=100)
Out[100]:
        0       1       2       3       4
0     0.0     2.0     4.0     6.0   104.0
1     9.0    11.0    13.0    15.0   109.0
2    18.0    20.0    22.0    24.0   114.0
3   115.0   116.0   117.0   118.0   119.0

In [101]: a.mul(b, fill_value=0)
Out[101]:
      0     1      2      3    4
0   0.0   1.0    4.0    9.0  0.0
1  20.0  30.0   42.0   56.0  0.0
2  80.0  99.0  120.0  143.0  0.0
3   0.0   0.0    0.0    0.0  0.0
```

fill_value参数替代NaN，替代后参与运算

数据类型的算术运算

```
In [102]: b = pd.DataFrame(np.arange(20).reshape(4,5))

In [103]: b
Out[103]:
    0   1   2   3   4
0   0   1   2   3   4
1   5   6   7   8   9
2  10  11  12  13  14
3  15  16  17  18  19

In [104]: c = pd.Series(np.arange(4))

In [105]: c
Out[105]:
0    0
1    1
2    2
3    3
dtype: int32
```

```
In [106]: c - 10
Out[106]:
0   -10
1    -9
2    -8
3    -7
dtype: int32

In [107]: b - c
Out[107]:
      0     1     2     3   4
0   0.0   0.0   0.0   0.0 NaN
1   5.0   5.0   5.0   5.0 NaN
2  10.0  10.0  10.0  10.0 NaN
3  15.0  15.0  15.0  15.0 NaN
```

不同维度间为广播运算，一维Series默认在轴1参与运算

数据类型的算术运算

```
In [102]: b = pd.DataFrame(np.arange(20).reshape(4,5))

In [103]: b
Out[103]:
    0   1   2   3   4
0   0   1   2   3   4
1   5   6   7   8   9
2  10  11  12  13  14
3  15  16  17  18  19

In [104]: c = pd.Series(np.arange(4))

In [105]: c
Out[105]:
0    0
1    1
2    2
3    3
dtype: int32
```

```
In [110]: b.sub(c, axis=0)
Out[110]:
    0   1   2   3   4
0   0   1   2   3   4
1   4   5   6   7   8
2   8   9  10  11  12
3  12  13  14  15  16
```

使用运算方法可以令一维Series参与轴0运算

Pandas库的数据排序

.sort_index()方法在指定轴上根据索引进行排序，默认升序

.sort_index(axis=0, ascending=True)

```
In [123]: b = pd.DataFrame(np.arange(20).reshape(4,5), index=['c','a','d','b'])

In [124]: b
Out[124]:
    0   1   2   3   4
c   0   1   2   3   4
a   5   6   7   8   9
d  10  11  12  13  14
b  15  16  17  18  19
```

```
In [125]: b.sort_index()
Out[125]:
    0   1   2   3   4
a   5   6   7   8   9
b  15  16  17  18  19
c   0   1   2   3   4
d  10  11  12  13  14

In [125]:

In [126]: b.sort_index(ascending=False)
Out[126]:
    0   1   2   3   4
d  10  11  12  13  14
c   0   1   2   3   4
b  15  16  17  18  19
a   5   6   7   8   9
```

Pandas库的数据排序

.sort_values()方法在指定轴上根据数值进行排序，默认升序

Series.sort_values(axis=0, ascending=True)

DataFrame.sort_values(by, axis=0, ascending=True)

by : axis轴上的某个索引或索引列表

Pandas库的数据排序

```
In [123]: b = pd.DataFrame(np.arange(20).reshape(4,5), index=['c','a','d','b'])

In [124]: b
Out[124]:
    0   1   2   3   4
c   0   1   2   3   4
a   5   6   7   8   9
d  10  11  12  13  14
b  15  16  17  18  19
```

```
In [129]: c = b.sort_values(2, ascending=False)

In [130]: c
Out[130]:
    0   1   2   3   4
b  15  16  17  18  19
d  10  11  12  13  14
a   5   6   7   8   9
c   0   1   2   3   4

In [131]: c = c.sort_values('a', axis=1, ascending=False)

In [132]: c
Out[132]:
    4   3   2   1   0
b  19  18  17  16  15
d  14  13  12  11  10
a   9   8   7   6   5
c   4   3   2   1   0
```

Pandas库的数据排序

NaN统一放到排序末尾

```
In [133]: a = pd.DataFrame(np.arange(12).reshape(3,4), index=['a','b','c'])

In [134]: a
Out[134]:
   0  1   2   3
a  0  1   2   3
b  4  5   6   7
c  8  9  10  11

In [135]: b = pd.DataFrame(np.arange(20).reshape(4,5), index=['c','a','d','b'])

In [136]: b
Out[136]:
    0   1   2   3   4
c   0   1   2   3   4
a   5   6   7   8   9
d  10  11  12  13  14
b  15  16  17  18  19

In [137]: c = a + b

In [138]: c
Out[138]:
      0     1     2     3   4
a   5.0   7.0   9.0  11.0 NaN
b  19.0  21.0  23.0  25.0 NaN
c   8.0  10.0  12.0  14.0 NaN
d   NaN   NaN   NaN   NaN NaN
```

```
In [139]: c.sort_values(2, ascending=False)
Out[139]:
      0     1     2     3   4
b  19.0  21.0  23.0  25.0 NaN
c   8.0  10.0  12.0  14.0 NaN
a   5.0   7.0   9.0  11.0 NaN
d   NaN   NaN   NaN   NaN NaN

In [140]: c.sort_values(2, ascending=True)
Out[140]:
      0     1     2     3   4
a   5.0   7.0   9.0  11.0 NaN
c   8.0  10.0  12.0  14.0 NaN
b  19.0  21.0  23.0  25.0 NaN
d   NaN   NaN   NaN   NaN NaN
```

Pandas库的基本统计分析

适用于Series和DataFrame类型

方法	说明
.sum()	计算数据的总和，按0轴计算，下同
.count()	非NaN值的数量
.mean() .median()	计算数据的算术平均值、算术中位数
.var() .std()	计算数据的方差、标准差
.min() .max()	计算数据的最小值、最大值

Pandas库的基本统计分析

适用于Series类型

方法	说明
.argmin() .argmax()	计算数据最大值、最小值所在位置的索引位置（自动索引）
.idxmin() .idxmax()	计算数据最大值、最小值所在位置的索引（自定义索引）

Pandas库的基本统计分析

适用于Series和DataFrame类型

方法	说明
.describe()	针对0轴（各列）的统计汇总

Pandas库的基本统计分析

```
In [141]: a = pd.Series([9, 8, 7, 6], index=['a', 'b', 'c', 'd'])

In [142]: a
Out[142]:
a    9
b    8
c    7
d    6
dtype: int64

In [143]: a.describe()
Out[143]:
count    4.000000
mean     7.500000
std      1.290994
min      6.000000
25%      6.750000
50%      7.500000
75%      8.250000
max      9.000000
dtype: float64
```

```
In [144]: type(a.describe())
Out[144]: pandas.core.series.Series

In [145]: a.describe()['count']
Out[145]: 4.0

In [146]: a.describe()['max']
Out[146]: 9.0
```

Pandas库的基本统计分析

```
In [147]: b = pd.DataFrame(np.arange(20).reshape(4,5), index=['c','a','d','b'])

In [148]: b.describe()
Out[148]:
              0          1          2          3          4
count   4.000000   4.000000   4.000000   4.000000   4.000000
mean    7.500000   8.500000   9.500000  10.500000  11.500000
std     6.454972   6.454972   6.454972   6.454972   6.454972
min     0.000000   1.000000   2.000000   3.000000   4.000000
25%     3.750000   4.750000   5.750000   6.750000   7.750000
50%     7.500000   8.500000   9.500000  10.500000  11.500000
75%    11.250000  12.250000  13.250000  14.250000  15.250000
max    15.000000  16.000000  17.000000  18.000000  19.000000

In [149]: type(b.describe())
Out[149]: pandas.core.frame.DataFrame
```

```
In [150]: b.describe().ix['max']
__main__:1: DeprecationWarning:
.ix is deprecated. Please use
.loc for label based indexing or
.iloc for positional indexing

See the documentation here:
http://pandas.pydata.org/pandas-docs/stable/indexing.html#ix-indexer-is-deprecated
Out[150]:
0    15.0
1    16.0
2    17.0
3    18.0
4    19.0
Name: max, dtype: float64

In [151]: b.describe()[2]
Out[151]:
count     4.000000
mean      9.500000
std       6.454972
min       2.000000
25%       5.750000
50%       9.500000
75%      13.250000
max      17.000000
Name: 2, dtype: float64

In [152]: b.describe().loc['max']
Out[152]:
0    15.0
1    16.0
2    17.0
3    18.0
4    19.0
Name: max, dtype: float64
```

Pandas库的累计统计分析

适用于Series和DataFrame类型，累计计算

方法	说明
.cumsum()	依次给出前1、 2、 …、 n个数的和
.cumprod()	依次给出前1、 2、 …、 n个数的积
.cummax()	依次给出前1、 2、 …、 n个数的最大值
.cummin()	依次给出前1、 2、 …、 n个数的最小值

Pandas库的累计统计分析

适用于Series和DataFrame类型，滚动计算（窗口计算）

方法	说明
.rolling(w).sum()	依次计算相邻w个元素的和
.rolling(w).mean()	依次计算相邻w个元素的算术平均值
.rolling(w).var()	依次计算相邻w个元素的方差
.rolling(w).std()	依次计算相邻w个元素的标准差
.rolling(w).min() .max()	依次计算相邻w个元素的最小值和最大值

相关分析

两个事物，表示为X和Y，如何判断它们之间的存在相关性？

相关性

- X增大，Y增大，两个变量正相关
- X增大，Y减小，两个变量负相关
- X增大，Y无视，两个变量不相关

协方差

两个事物，表示为X和Y，如何判断它们之间的存在相关性？

$$cov(X,Y) = \frac{\sum_{i=1}^{n}(X_i - \tilde{X})(Y_i - \tilde{Y})}{n-1}$$

- 协方差>0，X和Y正相关
- 协方差<0，X和Y负相关
- 协方差=0，X和Y独立无关

Pearson相关系数

两个事物，表示为X和Y，如何判断它们之间的存在相关性？

$$r = \frac{\sum_{i=1}^{n}(x_i - \tilde{x})(y_i - \tilde{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \tilde{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \tilde{y})^2}}$$

r取值范围[-1,1]

- 0.8-1.0 极强相关
- 0.6-0.8 强相关
- 0.4-0.6 中等程度相关
- 0.2-0.4 弱相关
- 0.0-0.2 极弱相关或无相关

Pandas库的累计统计分析

适用于Series和DataFrame类型

方法	说明
.cov()	计算协方差矩阵
.corr()	计算相关系数矩阵, Pearson、 Spearman、 Kendall等系数

Python库之数据分析

SciPy: 数学、科学和工程计算功能库

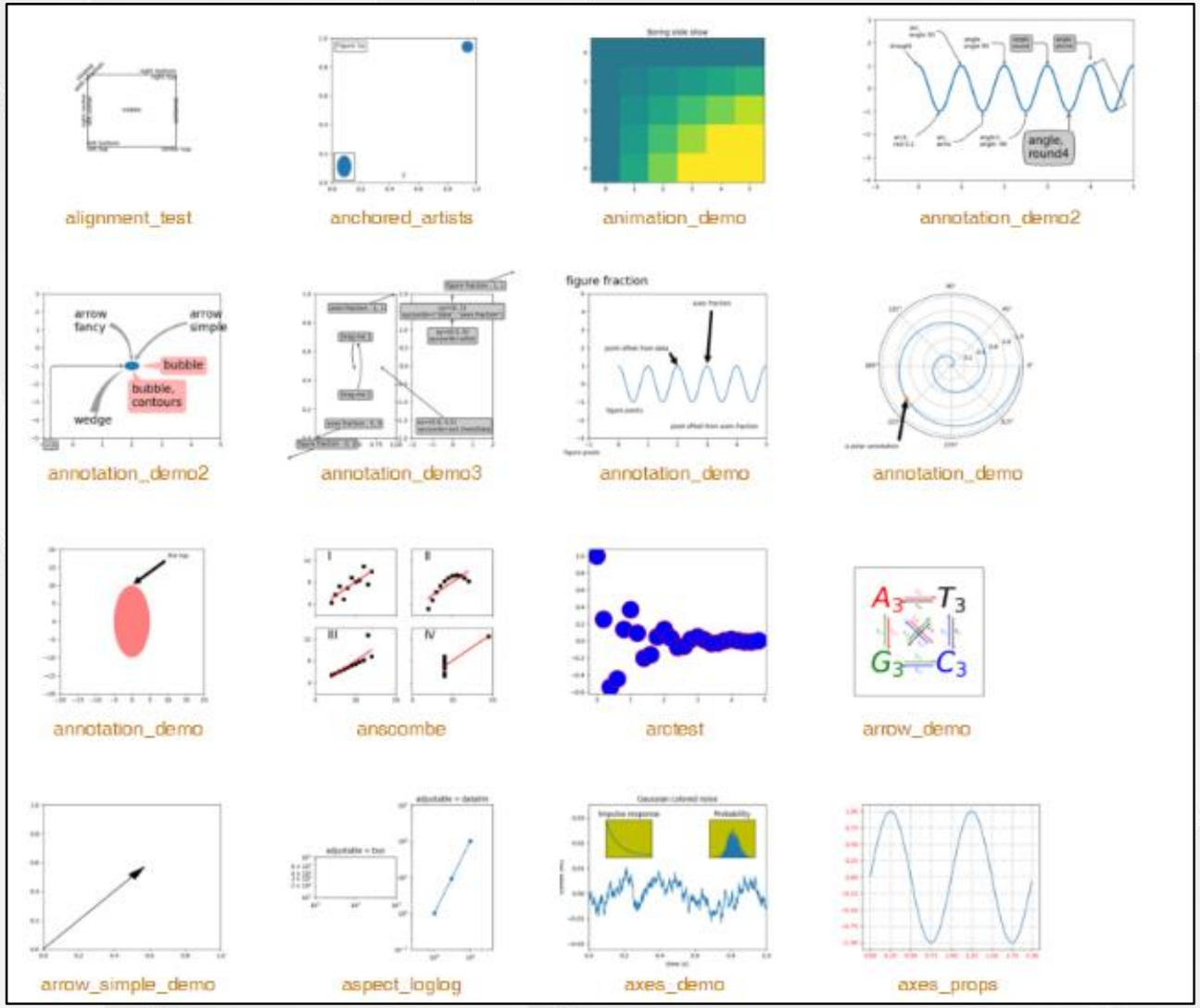


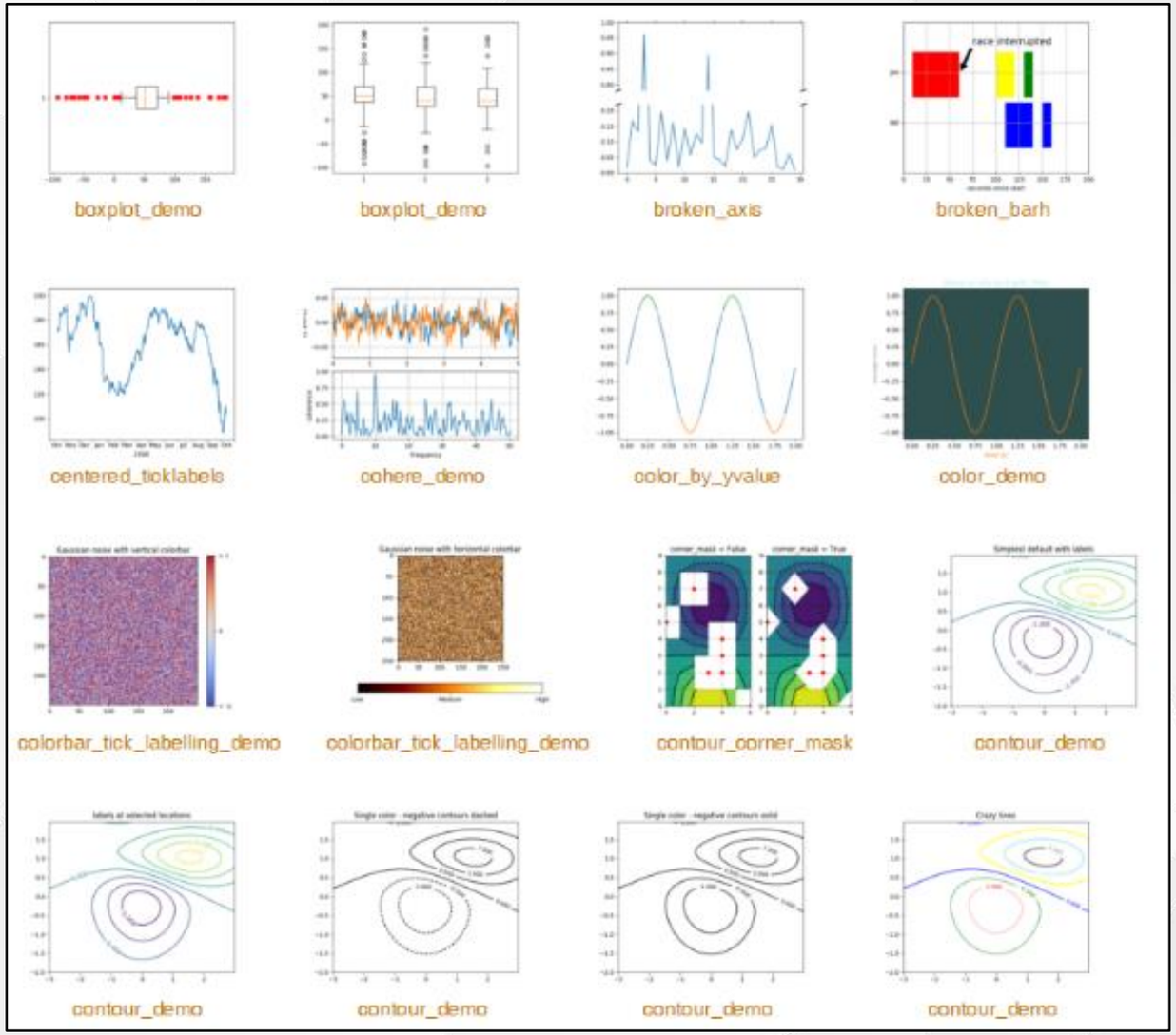
http://www.scipy.org

从数据处理到人工智能

- Python库之数据分析
- Python库之数据可视化
- Python库之文本处理
- Python库之机器学习

Matplotlib库的效果





http://matplotlib.org

Matplotlib库的使用

Matplotlib库由各种可视化类构成，内部结构复杂，受Matlab启发

matplotlib.pyplot是绘制各类可视化图形的命令子库，相当于快捷方式

```
import matplotlib.pyplot as plt
```

引入的模块名

pyplot的plot()函数

```
plt.plot(x, y, format_string, **kwargs)
```

- x：X轴数据，列表或数组，可选
- y：Y轴数据，列表或数组
- format_string: 控制曲线的格式字符串，可选
- **kwargs ：第二组或更多(x,y,format_string)

当绘制多条曲线时，各条曲线的x不能省略

pyplot的plot()函数

```
plt.plot(x, y, format_string, **kwargs)
```

**kwargs: 第二组或更多(x, y, format_string)

- color：控制颜色, color='green'
- linestyle：线条风格, linestyle='dashed'
- marker ：标记风格, marker='o'
- markerfacecolor ：标记颜色, markerfacecolor='blue'
- markersize ：标记尺寸, markersize=20

……

pyplot的绘图区域

```
plt.subplot(nrows, ncols, plot_number)
```

```
plt.subplot(3,2,4)
plt.subplot(324)
```

在全局绘图区域中创建一个分区体系，并定位到一个子绘图区域

pyplot的plot()函数

```
plt.plot(x, y, format_string, **kwargs)
```

format_string: 控制曲线的格式字符串，可选

由颜色字符、 风格字符和标记字符组成

颜色字符	说明	颜色字符	说明
'b'	蓝色	'm'	洋红色 magenta
'g'	绿色	'y'	黄色
'r'	红色	'k'	黑色
'c'	青绿色 cyan	'w'	白色
'#008000'	RGB某颜色	'0.8'	灰度值字符串

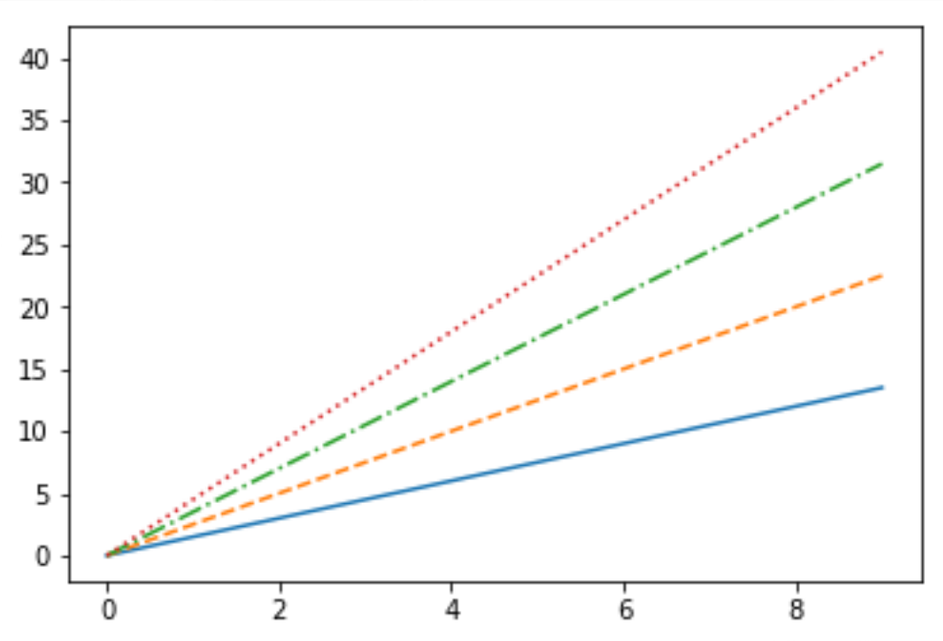
pyplot的plot()函数

```
plt.plot(x, y, format_string, **kwargs)
```

format_string: 控制曲线的格式字符串，可选

由颜色字符、 风格字符和标记字符组成

风格字符	说明
'-'	实线
'--'	破折线
'-.'	点划线
':'	虚线
'' ' '	无线条



pyplot的plot()函数

format_string: 控制曲线的格式字符串，可选

由颜色字符、 风格字符和标记字符组成

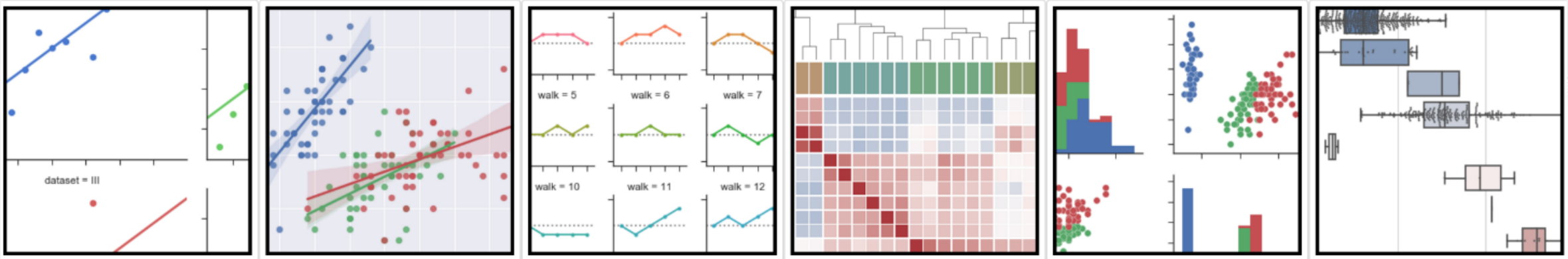
标记字符	说明	标记字符	说明	标记字符	说明
'.'	点标记	'1'	下花三角标记	'h'	竖六边形标记
','	像素标记(极小点)	'2'	上花三角标记	'H'	横六边形标记
'o'	实心圈标记	'3'	左花三角标记	'+'	十字标记
'v'	倒三角标记	'4'	右花三角标记	'x'	x标记
'^'	上三角标记	's'	实心方形标记	'D'	菱形标记
'>'	右三角标记	'p'	实心五角标记	'd'	瘦菱形标记
'<'	左三角标记	'*'	星形标记	'|'	垂直线标记

pyplot的文本显示函数

函数	说明
plt.xlabel()	对X轴增加文本标签
plt.ylabel()	对Y轴增加文本标签
plt.title()	对图形整体增加文本标签
plt.text()	在任意位置增加文本
plt.annotate()	在图形中增加带箭头的注解

Python库之数据可视化

Seaborn: 统计类数据可视化功能库



http://seaborn.pydata.org/

Python库之数据可视化

pyecharts: python版本的Echarts库（百度开源的数据可视化图表库）



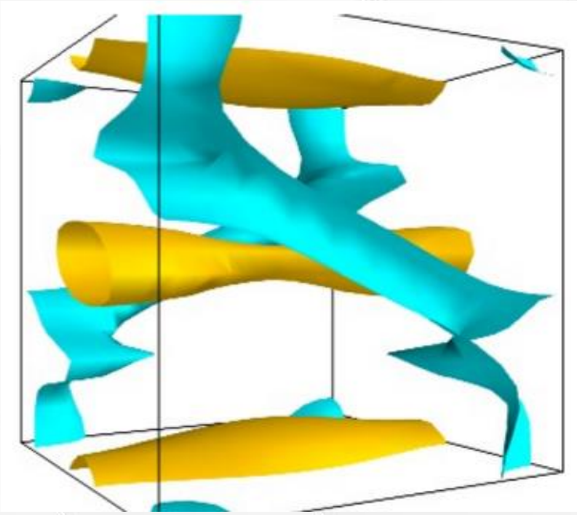
pip install pyecharts

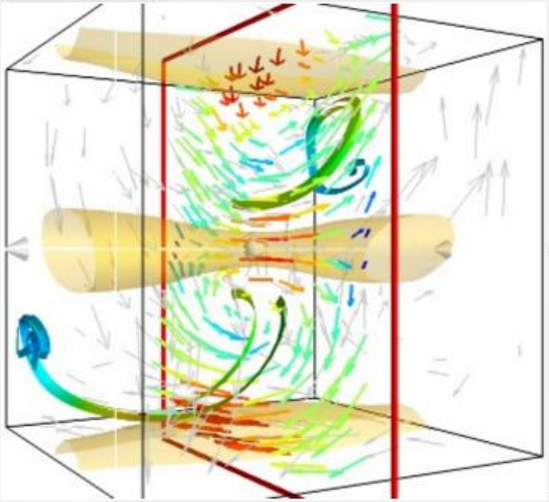
https://pyecharts.org/#/
https://echarts.apache.org/examples/en/index.html

Python库之数据可视化

Mayavi：三维科学数据可视化功能库







http://docs.enthought.com/mayavi/mayavi/

从数据处理到人工智能

- Python库之数据分析
- Python库之数据可视化
- Python库之文本处理
- Python库之机器学习

Python库之文本处理

PyPDF2：用来处理pdf文件的工具集

- 提供了一批处理PDF文件的计算功能
- 支持获取信息、分隔/整合文件、加密解密等
- 完全Python语言实现，不需要额外依赖，功能稳定

http://mstamy2.github.io/PyPDF2

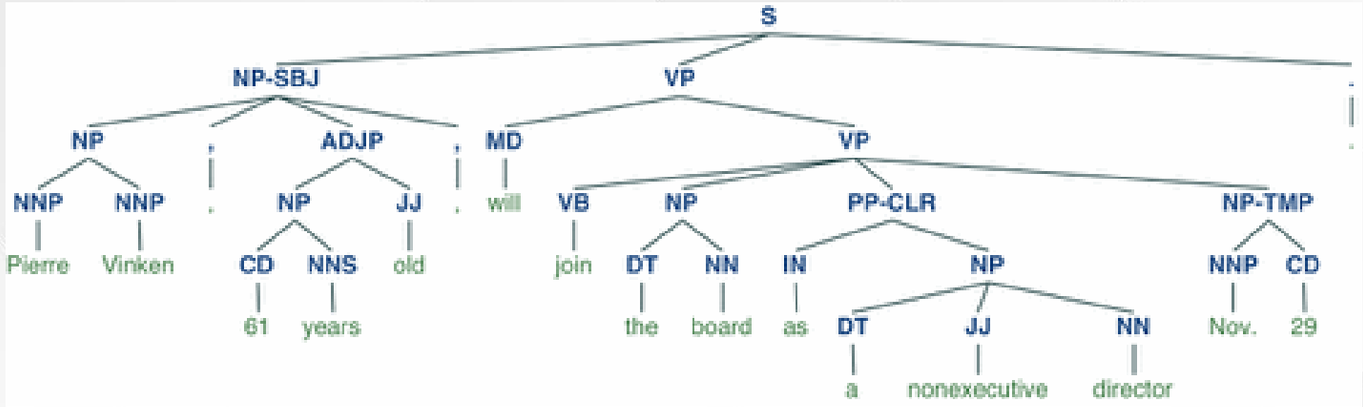
Python库之文本处理

NLTK：自然语言文本处理第三方库

```
from nltk.corpus import treebank

t = treebank.parsed_sents('wsj_0001.mrg')[0]

t.draw()
```



http://www.nltk.org/

Python库之文本处理

python-docx：创建或更新Microsoft Word文件的第三方库

- 提供创建或更新.doc .docx等文件的计算功能
- 增加并配置段落、图片、表格、文字等，功能全面

http://python-docx.readthedocs.io/en/latest/index.html

Python库之文本处理

jieba库：中文分词库

中文分词是文本分析的基础搜索引擎、机器翻译、语音合成、自动分类、自动摘要、自动校对等都需要用到分词

分词是对一句话中的文字进行切分，将其分成不同的词

```
pip install jieba
```

https://github.com/fxsjy/jieba

jieba库

函数	描述
jieba.cut(s)	精确模式，返回一个可迭代的数据类型
jieba.cut(s, cut_all=True)	全模式，输出文本s中所有可能单词
jieba.cut_for_search(s)	搜索引擎模式，适合搜索引擎建立索引的分词结果
jieba.lcut(s)	精确模式，返回一个列表类型
jieba.lcut(s, cut_all=True)	全模式，返回一个列表类型
jieba.lcut_for_search(s)	搜索引擎模式，返回一个列表类型
jieba.add_word(w)	向分词词典中增加新词w

jieba库

```
import jieba

source = '来到杨过曾经生活的地方，小龙女动情的说，我也想过过过儿过过的生活。’

jieba.add_word('过过') # 加入自定义词

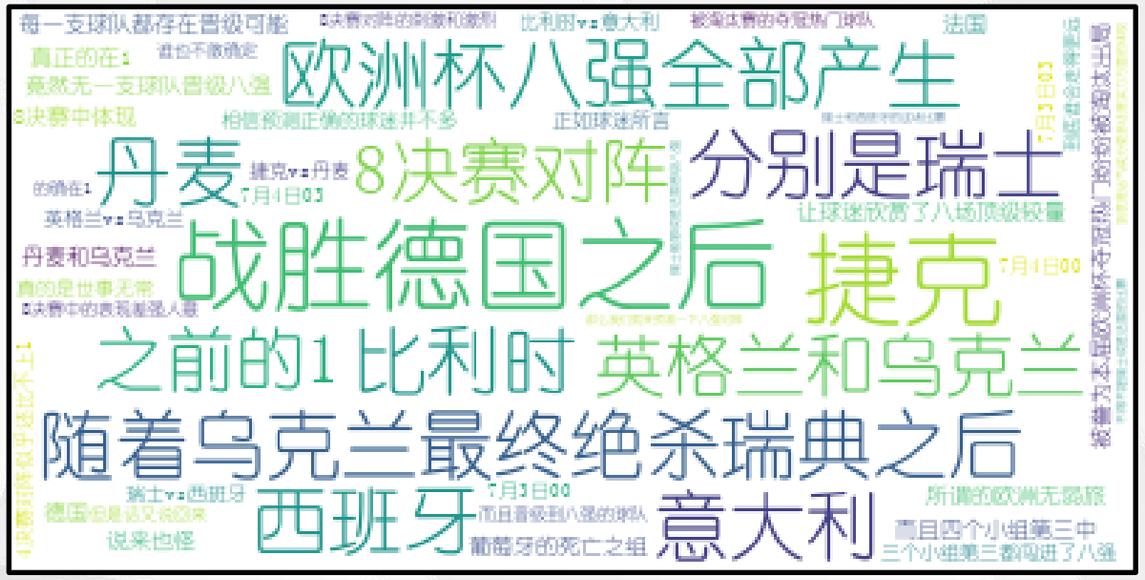
print("|".join(jieba.cut(source)))                    # 精确模式
print("|".join(jieba.cut(source, cut_all=True)))      # 全模式
print("|".join(jieba.cut_for_search(source)))         # 搜索引擎模式
```

```
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\sj\AppData\Local\Temp\jieba.cache
Loading model cost 0.520 seconds.
Prefix dict has been built successfully.
"|来到|杨过|曾经|生活|的|地方|，|小龙女|动情|的|说|，|我|也|想|过过|过儿|过过|的|生活|。
"|来到|杨|过|曾经|生活|的|地方|，|小龙|小龙女|龙女|动情|的|说|，|我|也|想|过过|过过|过儿|过过|的|生活|。
"|来到|杨过|曾经|生活|的|地方|，|小龙|龙女|小龙女|动情|的|说|，|我|也|想|过过|过儿|过过|的|生活|。
```

Python库之文本处理

wordcloud库：词云库

pip install wordcloud





https://github.com/amueller/word_cloud

wordcloud库

```
from wordcloud import WordCloud

# 读取文件
with open('./data/欧洲杯半决赛对阵预测.txt', 'r', encoding='utf-8', errors='ignore') as f:
    txt = f.read()

# 创建WordCloud对象
wc = WordCloud(font_path='SIMYOU.TTF',
               mode='RGBA',
               background_color='white')

# 从文本生成词云
wc.generate(txt)

# 将词云保存到文件
wc.to_file('./data/wordcloud.png')
```

导入词云库

指定字体文件的路径，必须选择合适的字体，否则无法正常显示

由文本生成词云，但此时并没有生成图片文件

保存词云到文件

```python
from wordcloud import WordCloud

import matplotlib.pyplot as plt
from PIL import Image
import numpy as np

# 读取文件
with open('./data/欧洲杯半决赛对阵预测.txt', 'r', encoding='utf-8', errors='ignore') as f:
    txt = f.read()

custom_mask = np.array(Image.open("./data/wor
# 创建WordCloud对象
wc = WordCloud(font_path='SIMYOU.TTF',
               mode='RGBA',
               mask=custom_mask,
               background_color='white')

# 从文本生成词云
wc.generate(txt)

# 将词云保存到文件
# wc.to_file('./data/wordcloud.png')

# 绘制词云图
plt.imshow(wc)
plt.axis('off')
# 显示生成词云图
plt.show()
```

导入词云库

导入matplotlib库中的matplotlib.pyplot模块

导入pillow库中的PIL模块、numpy库

它是
y数组



从数据处理到人工智能

- Python库之数据分析
- Python库之数据可视化
- Python库之文本处理
- Python库之机器学习

Python库之机器学习

Scikit-learn：机器学习方法工具集

- 提供一批统一化的机器学习方法功能接口
- 提供聚类、分类、回归、强化学习等计算功能
- 机器学习最基本且最优秀的Python第三方库

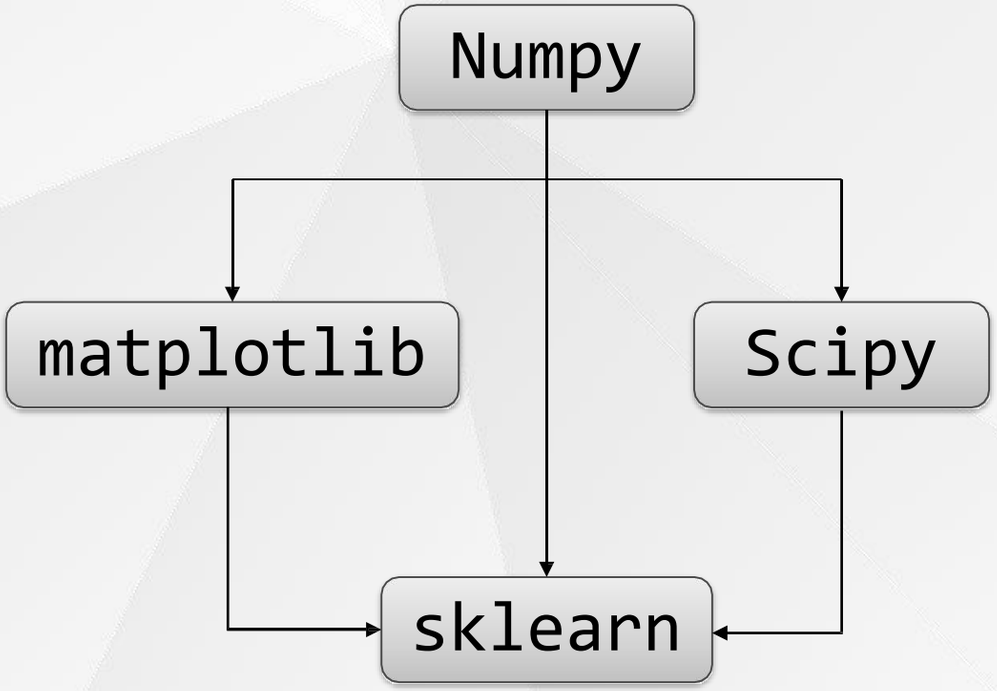
```
pip install scikit-learn
```

http://scikit-learn.org/

Scikit-learn库

安装顺序如下：

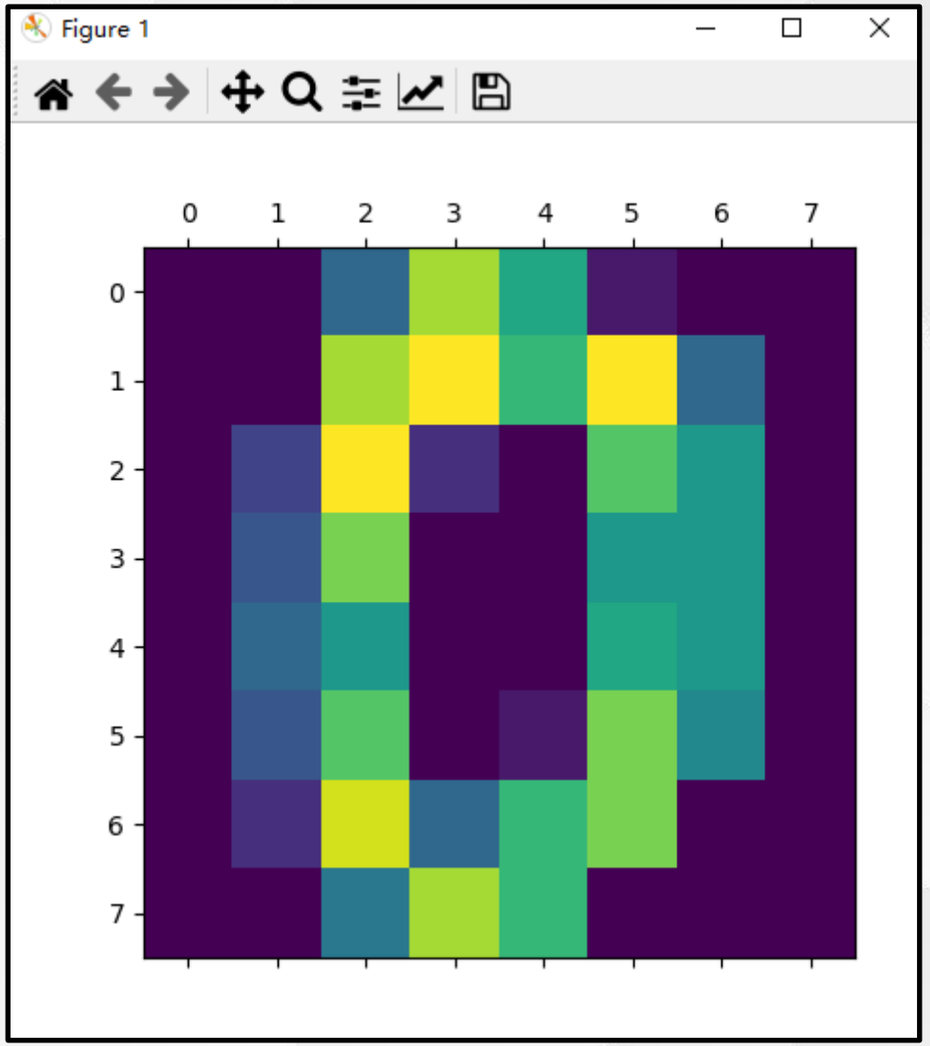
1. Numpy库
2. Scipy库
3. matplotlib库
4. sklearn库



Scikit-learn库的标准数据集

示例：

```
>>> from sklearn.datasets import load_digits
>>> digits = load_digits()
>>> print(digits.data.shape)
(1797, 64)
>>> print(digits.target.shape)
(1797, )
>>> print(digits.images.shape)
(1797, 8, 8)
>>> import matplotlib.pyplot as plt
>>> plt.matshow(digits.images[0])
>>> plt.show()
```



Python库之机器学习

TensorFlow：AlphaGo背后的机器学习计算框架

- 谷歌公司推动的开源机器学习框架
- 将数据流图作为基础，图节点代表运算，边代表张量
- 应用机器学习方法的一种方式，支撑谷歌人工智能应用

https://www.tensorflow.org/

Python库之机器学习

PyTorch： Facebook公司推动的机器学习计算框架

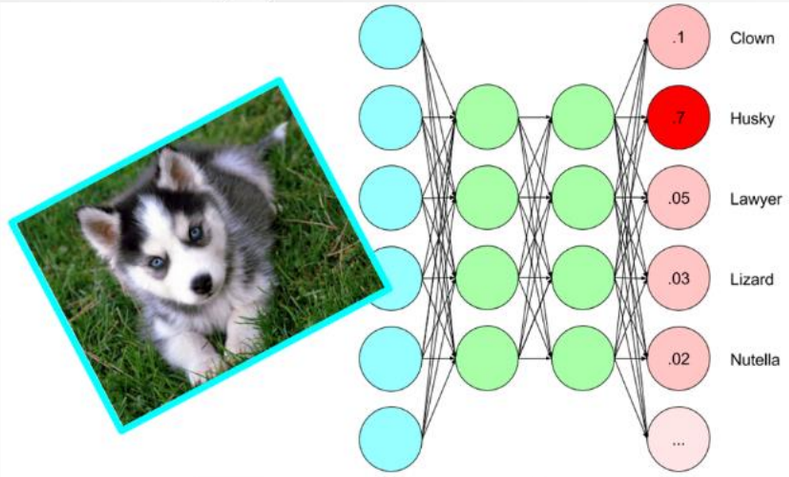
- 与Tensorflow的静态计算图不同，pytorch的计算图是动态的，可以根据计算需要实时改变计算图
- 应用机器学习方法的一种方式



https://pytorch.org/

Python库之机器学习

MXNet：基于神经网络的深度学习计算框架



mxnet

https://mxnet.incubator.apache.org/

Python库之机器学习



OpenCV/Pillow：图像处理库



- OpenCV支持C++/Python图像处理库
- Pillow支持Python
- 可用于自动驾驶、机器翻译、语音识别等众多领域

https://opencv.org/
https://python-pillow.org/

图像的RGB色彩模式

RGB三个颜色通道的变化和叠加得到各种颜色，其中

- R 红色，取值范围，0-255
- G 绿色，取值范围，0-255
- B 蓝色，取值范围，0-255

RGB形成的颜色包括了人类视力所能感知的所有颜色

OpenCV库

计算机视觉库

开源、支持多平台、多种编程语言

```
pip install opencv-python
```

OpenCV库

```
import cv2

img = cv2.imread('./images/Lenna.jpg')

print('图像属性：')
print('img对象类型：', type(img))
print('数组形状：', img.shape)
# 获得图像的高度
h = img.shape[0]
# 获得图像的宽度
w = img.shape[1]

msg = '''
图像高度：  {0}
图像宽度：  {1}
图像总像素数：  {2}
'''

print(msg.format(h, w, img.size))

cv2.imshow('Lenna', img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

导入OpenCV库

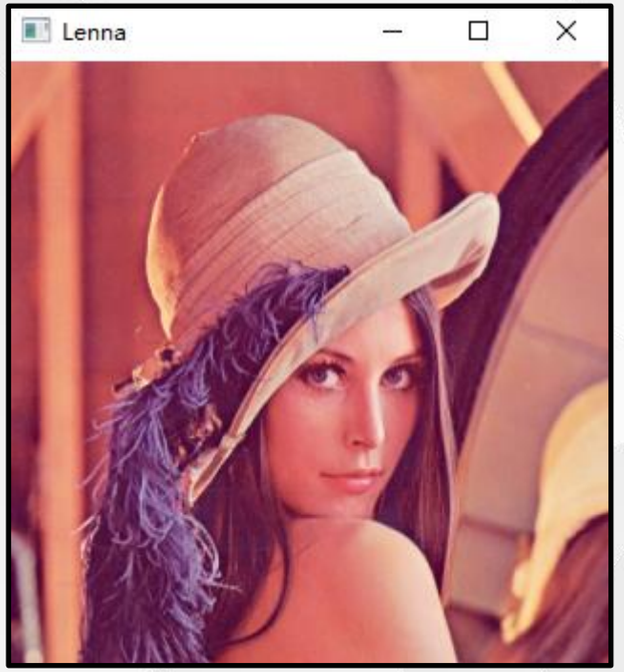
Imread函数读取images目录下的lenna.jpg图像文件，返回图像对象

type函数返回图像对象的数据类型，即ndarray类型

shape属性返回图像的形状

```
图像属性：
img对象类型：  <class 'numpy.ndarray'>
数组形状：  (316, 316, 3)

图像高度：  316
图像宽度：  316
图像总像素数：  299568
```



waitKey函数用于等待用户按任意键n毫秒，若为0，则表示一直等待，指导用户按下任意键

关闭并释放所有窗口

PIL库

PIL, Python Image Library

PIL库是一个具有强大图像处理能力的第三方库

在命令行下的安装方法：pip install pillow

```
from PIL import Image
```

Image是PIL库中代表一个图像的类（对象）

图像的数组表示

图像是一个三维数组，维度分别是高度、宽度和像素RGB值

```
In [210]: from PIL import Image

In [211]: import numpy as np

In [212]: im = np.array(Image.open("G:/beijing.jpg"))

In [213]: print(im.shape, im.dtype)
(669, 1012, 3) uint8
```

图像的变换

```
In [219]: from PIL import Image
     ...: import numpy as np
     ...: im = np.array(Image.open("G:/beijing.jpg").convert('L'))
     ...: b = 255 - im
     ...: c = Image.fromarray(b.astype('uint8'))
     ...: c.save("G:/beijing3.jpg")
```





02

从Web解析到网络空间

从Web解析到网络空间

- Python库之网络爬虫
- Python库之Web信息提取
- Python库之Web网站开发
- Python库之网络应用开发

Python库之网络爬虫

Requests: 最友好的网络爬虫功能库

- 提供了简单易用的类HTTP协议网络爬虫功能
- 支持连接池、SSL、Cookies、HTTP(S)代理等
- Python最主要的页面级网络爬虫功能库

```
pip install requests
```

Requests库的7个主要方法

方法	说明
requests.request()	构造一个请求，支撑以下各方法的基础方法
requests.get()	获取HTML网页的主要方法，对应于HTTP的GET
requests.head()	获取HTML网页头信息的方法，对应于HTTP的HEAD
requests.post()	requests.post() 向HTML网页提交POST请求的方法，对应于HTTP的POST
requests.put()	向HTML网页提交PUT请求的方法，对应于HTTP的PUT
requests.patch()	向HTML网页提交局部修改请求，对应于HTTP的PATCH
requests.delete()	向HTML页面提交删除请求，对应于HTTP的DELETE

Requests库的get()方法

```
requests.get()
```

```
r = requests.get(url)
```



Response对象包含爬虫返回的内容

Response对象

```
In [230]: import requests

In [231]: r = requests.get("http://www.baidu.com")

In [232]: print(r.status_code)
200

In [233]: type(r)
Out[233]: requests.models.Response

In [234]: r.headers
Out[234]: {'Cache-Control': 'private, no-cache, no-store, proxy-revalidate, no-transform', 'Connection':
'keep-alive', 'Content-Encoding': 'gzip', 'Content-Type': 'text/html', 'Date': 'Sat, 09 Oct 2021 04:36:28
GMT', 'Last-Modified': 'Mon, 23 Jan 2017 13:27:36 GMT', 'Pragma': 'no-cache', 'Server': 'bfe/1.0.8.18',
'Set-Cookie': 'BDORZ=27315; max-age=86400; domain=.baidu.com; path=/', 'Transfer-Encoding': 'chunked'}
```

Response对象包含服务器返回的所有信息，也包含请求的Request信息

Response对象的属性

属性	说明
r.status_code	HTTP请求的返回状态，200表示连接成功，404表示失败
r.text	HTTP响应内容的字符串形式，即 url对应的页面内容
r.encoding	从HTTP header中猜测的响应内容编码方式
r.apparent_encoding	从内容中分析出的响应内容编码方式（备选编码方式）
r.content	HTTP响应内容的二进制形式

理解Response的编码

r.encoding	从HTTP header中猜测的响应内容编码方式
r.apparent_encoding	从内容中分析出的响应内容编码方式（备选编码方式）

r.encoding：如果header中不存在charset，则认为编码为ISO-8859-1

r.text根据r.encoding显示网页内容

r.apparent_encoding：根据网页内容分析出的编码方式

可以看作是r.encoding的备选

理解Response的异常

```
r = requests.get(url)
```



Exception

网络连接有风险，异常处理很重要

理解Response的异常

异常	说明
requests.ConnectionError	网络连接错误异常，如DNS查询失败、拒绝连接等
requests.HTTPError	HTTP错误异常
requests.URLRequired	URL缺失异常
requests.TooManyRedirects	超过最大重定向次数，产生重定向异常
requests.ConnectTimeout	连接远程服务器超时异常
requests.Timeout	requests.Timeout 请求URL超时，产生超时异常

理解Response的异常

r.raise_for_status()	如果不是200，产生异常 requests.HTTPError

```
r = requests.get(url)
```

r.raise_for_status()在方法内部判断r.status_code是否等于200，不需要增加额外的if语句，该语句便于利用try-except进行异常处理

爬取网页的通用代码框架

```
In [246]: def getHTMLText(url):
     ...:     try:
     ...:         r = requests.get(url, timeout=30)
     ...:         r.raise_for_status()
     ...:         r.encoding = r.apparent_encoding
     ...:         return r.text
     ...:     except:
     ...:         return "产生异常"
     ...:

In [247]: url = "http://www.baidu.com"
     ...: print(getHTMLText(url))
     ...:
     ...:
<!DOCTYPE html>
<!--STATUS OK--><html> <head><meta http-equiv=content-type content=text/
html;charset=utf-8><meta http-equiv=X-UA-Compatible content=IE=Edge><meta
content=always name=referrer><link rel=stylesheet type=text/css
href=http://s1.bdstatic.com/r/www/cache/bdorz/baidu.min.css><title>百度一
下，你就知道</title></head> <body link=#0000cc> <div id=wrapper> <div
id=head> <div class=head_wrapper> <div class=s_form> <div
class=s_form_wrapper> <div id=lg> <img hidefocus=true src=//www.baidu.com/
img/bd_logo1.png width=270 height=129> </div> <form id=form name=f
action=//www.baidu.com/s class=fm> <input type=hidden name=bdorz_come
value=1> <input type=hidden name=ie value=utf-8> <input type=hidden name=f
value=8> <input type=hidden name=rsv_bp value=1> <input type=hidden
name=rsv_idx value=1> <input type=hidden name=tn value=baidu><span
class="bg s_ipt_wr"><input id=kw name=wd class=s_ipt value maxlength=255
```

如果状态不是200，引发HTTPError异常

HTTP协议

HTTP，Hypertext Transfer Protocol，超文本传输协议

HTTP是一个基于“请求与响应”模式的、无状态的应用层协议

HTTP协议采用URL作为定位网络资源的标识，URL格式如下：

```
http://host[:port][path]
```

host：合法的Internet主机域名或IP地址

port：端口号，缺省端口为80

path：请求资源的路径

HTTP协议

HTTP, Hypertext Transfer Protocol, 超文本传输协议

HTTP URL实例:

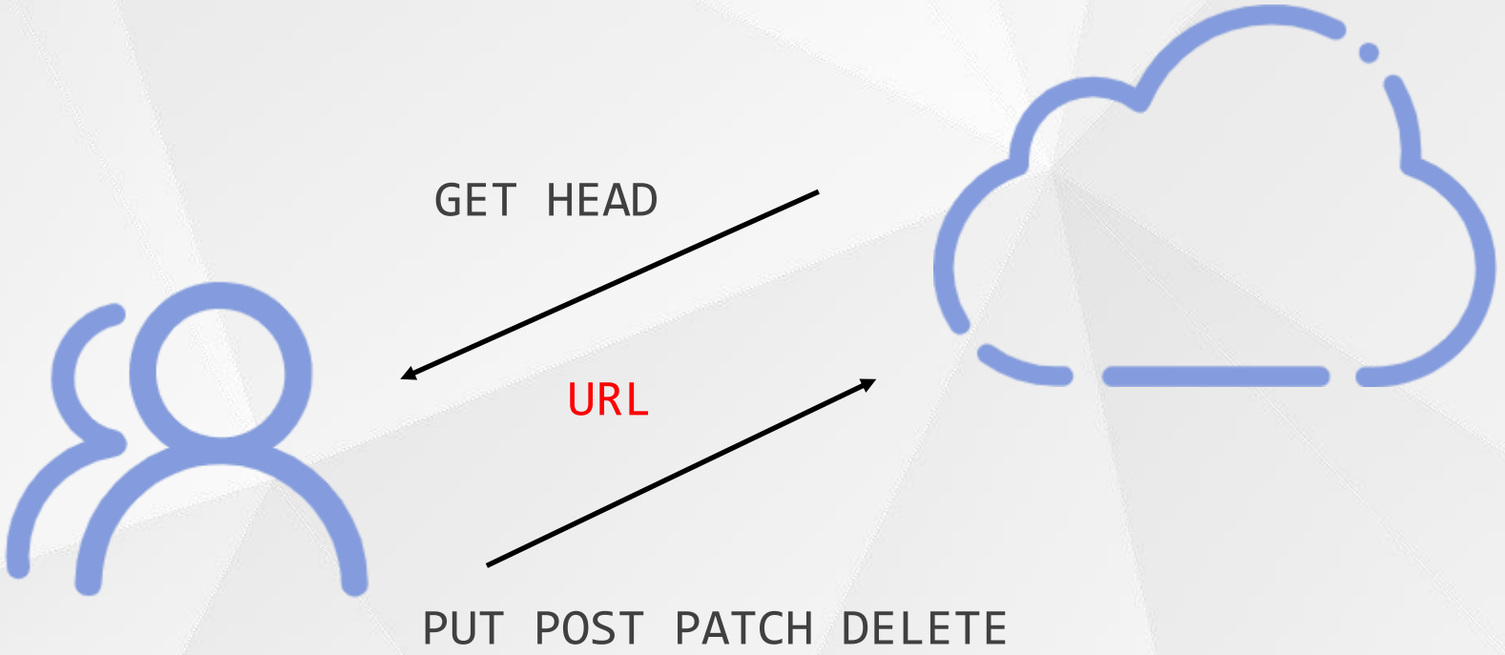
http://www.bit.edu.cn

http://220.181.111.188/duty

HTTP URL的理解:

URL是通过HTTP协议存取资源的Internet路径, 一个URL对应一个数据资源

HTTP协议对资源的操作



通过URL和命令管理资源，操作独立无状态，网络通道及服务器成为了黑盒子

HTTP协议与Requests库

HTTP协议方法	Requests库方法	说明
GET	requests.get()	获取HTML网页的主要方法，对应于HTTP的GET
HEAD	requests.head()	获取HTML网页头信息的方法，对应于HTTP的HEAD
POST	requests.post()	requests.post() 向HTML网页提交POST请求的方法，对应于HTTP的POST
PUT	requests.put()	向HTML网页提交PUT请求的方法，对应于HTTP的PUT
PATCH	requests.patch()	向HTML网页提交局部修改请求，对应于HTTP的PATCH
DELETE	requests.delete()	向HTML页面提交删除请求，对应于HTTP的DELETE

京东商品页面的爬取

https://item.jd.com/100583492850 57.html




```
import requests

url = "https://item.jd.com/100583492850 57.html"

try:
    r = requests.get(url)
    r.raise_for_status()
    r.encoding = r.apparent_encoding
    print(r.text[:1000])
except:
    print("爬取失败")
```

图片爬取

http://www.hfut.edu.cn/__local/F/BF/24/B5D33A17D42A95F5B31A48E9D51_9F0B79AE_197F6.jpg



```
import requests
import os
url =
"http://www.hfut.edu.cn/__local/F/BF/24/B5D33A17D42A95F5B
31A48E9D51_9F0B79AE_197F6.jpg"
root = "D://testpy//"
path = root + url.split('/')[-1]
try:
    if not os.path.exists(root):
        os.mkdir
    if not os.path.exists(path):
        r = requests.get(url)
        with open(path, 'wb') as f:
            f.write(r.content)
            f.close()
            print("文件保存成功")
    else:
        print("文件已存在")
except:
    print("爬取失败")
```

Python库之网络爬虫

Scrapy: 优秀的网络爬虫框架

- 提供了构建网络爬虫系统的框架功能，功能半成品
- 支持批量和定时网页爬取、提供数据处理流程等
- Python最主要且最专业的网络爬虫框架

Python库之网络爬虫

pyspider: 强大的Web页面爬取系统

- 提供了完整的网页爬取系统构建功能
- 支持数据库后端、消息队列、优先级、分布式架构等
- Python重要的网络爬虫类第三方库

从Web解析到网络空间

- Python库之网络爬虫
- Python库之Web信息提取
- Python库之Web网站开发
- Python库之网络应用开发

Python库之Web信息提取

Beautiful Soup: HTML和XML的解析库

- 提供了解析HTML和XML等Web信息的功能
- 又名beautifulsoup4或bs4，可以加载多种解析引擎
- 常与网络爬虫库搭配使用，如Scrapy、requests等

https://www.crummy.com/software/BeautifulSoup/bs4

Python库之Web信息提取

Re: 正则表达式解析和处理功能库

regular expression, regex, RE

正则表达式是用来简洁表达一组字符串的表达式

```
'PN'
'PYN'
'PYTN'
'PYTHN'
'PYTHON'
```

一组字符串



正则表达式:

P(Y|YT|YTH|YTHO)?N

简洁表达

https://docs.python.org/3.6/library/re.html

Python库之Web信息提取

Python-Goose: 提取文章类型Web页面的功能库

- 提供了对Web页面中文章信息/视频等元数据的提取功能
- 针对特定类型Web页面，应用覆盖面较广
- Python最主要的Web信息提取库

https://github.com/grangier/python-goose

从Web解析到网络空间

- Python库之网络爬虫
- Python库之Web信息提取
- Python库之Web网站开发
- Python库之网络应用开发

Python库之Web网站开发

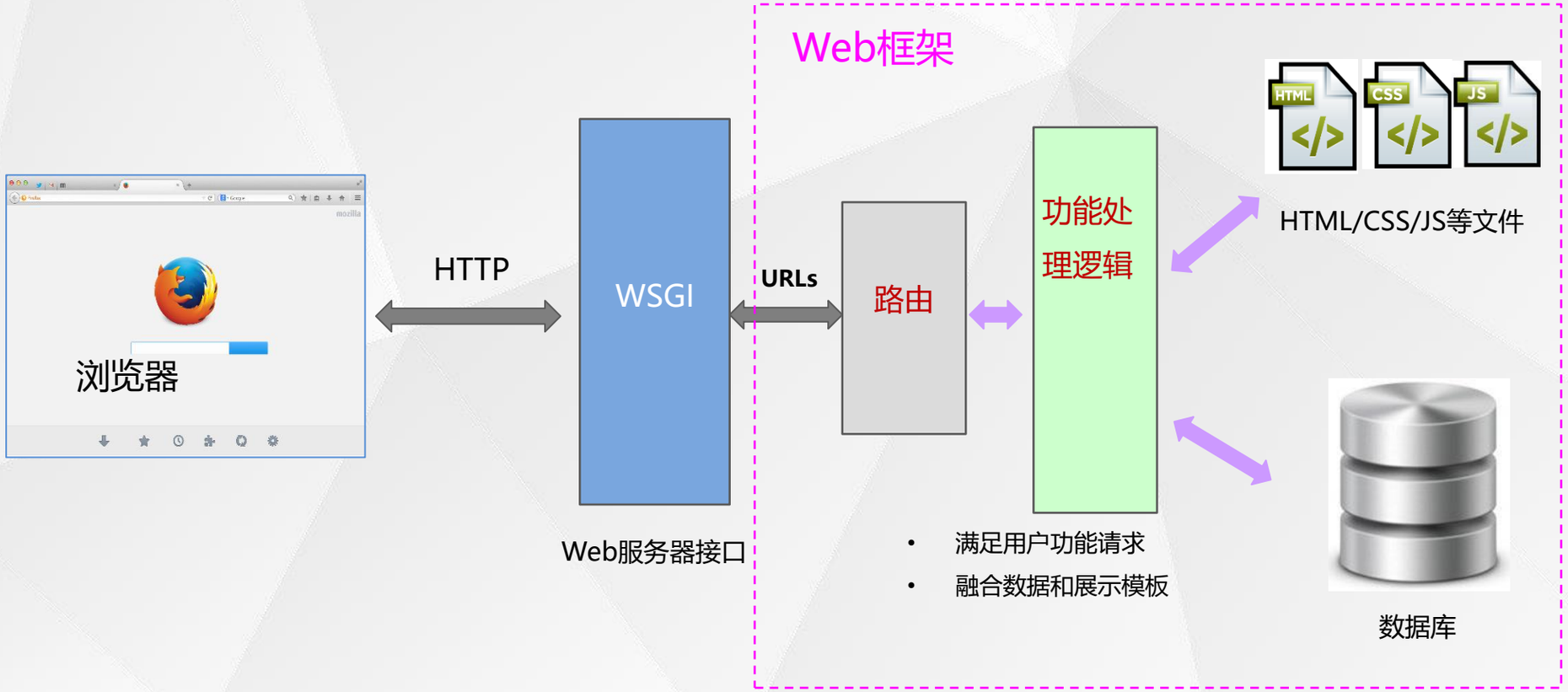
Django: 最流行的Web应用框架

- 提供了构建Web系统的基本应用框架
- MTV模式：模型(model)、模板(Template)、视图(Views)
- Python最重要的Web应用框架，略微复杂的应用框架

https://www.djangoproject.com

Python库之Web网站开发

Django: 最流行的Web应用框架



https://www.djangoproject.com

Python库之Web网站开发

Pyramid: 规模适中的Web应用框架

- 提供了简单方便构建Web系统的应用框架
- 不大不小，规模适中，适合快速构建并适度扩展类应用
- Python产品级Web应用框架，起步简单可扩展性好

https://trypyramid.com/

Python库之Web网站开发

Flask: Web应用开发微框架

- 提供了最简单构建Web系统的应用框架
- 特点是：简单、规模小、快速
- Django > Pyramid > Flask



http://flask.pocoo.org

从Web解析到网络空间

- Python库之网络爬虫
- Python库之Web信息提取
- Python库之Web网站开发
- Python库之网络应用开发

Python库之网络应用开发

WeRoBot: 微信公众号开发框架

- 提供了解析微信服务器消息及反馈消息的功能
- 建立微信机器人的重要技术手段

https://github.com/offu/WeRoBot

Python库之网络应用开发

aip: 百度AI开放平台接口

- 提供了访问百度AI服务的Python功能接口
- 语音、人脸、OCR、NLP、知识图谱、图像搜索等领域
- Python百度AI应用的最主要方式

Python库之网络应用开发

MyQR: 二维码生成第三方库

- 提供了生成二维码的系列功能
- 基本二维码、艺术二维码和动态二维码

03

从人机交互到艺术设计

从人机交互到艺术设计

- Python库之图形用户界面
- Python库之游戏开发
- Python库之虚拟现实
- Python库之图形艺术

Python库之图形用户界面

PyQt5: Qt开发框架的Python接口

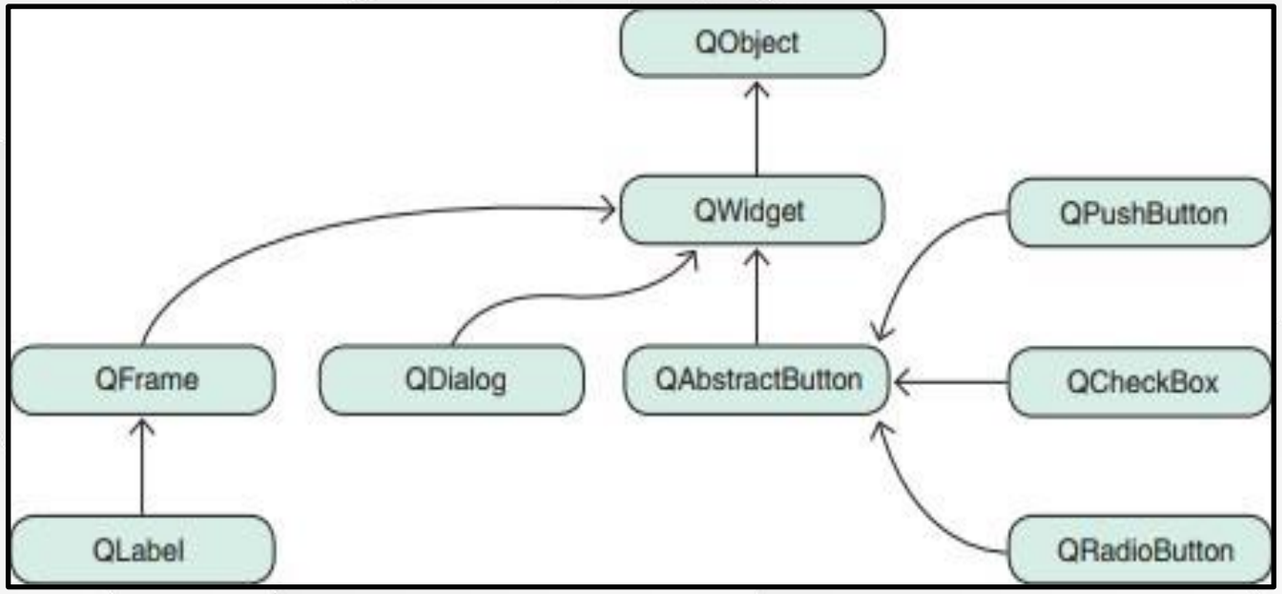
- 提供了创建Qt5程序的Python API接口
- Qt是非常成熟的跨平台桌面应用开发系统，完备GUI
- 推荐的Python GUI开发第三方库

```
pip install PyQt5
```

Python库之图形用户界面

PyQt5: Qt开发框架的Python接口

- 窗口
- 控件（Widget）
- 事件处理
- 布局管理



PyQt5

```
import sys
from PyQt5.QtWidgets import *


class MyWindow(QWidget):

    def __init__(self):
        super().__init__()

        self.initUI()

    def initUI(self):
        self.resize(400, 300)                    # 设置整窗口的大小
        self.move(600, 300)                      # 设置窗口的位置

        self.setWindowTitle('添加控件! ')          # 设置窗口标题

        label = QLabel(self)                     # 创建标签控件
        label.setText("Hello World!")            # 设置标签显示的文字
        label.move(180, 120)                     # 设置标签位置

        button = QPushButton(self)               # 创建按钮控件
        button.setText("OK")                     # 设置按钮上显示的文字
        button.move(170, 160)                    # 设置按钮位置


def main():
    app = QApplication(sys.argv)                 # 创建应用程序对象
    window = MyWindow()

    window.show()                                # 显示窗口
    sys.exit(app.exec_())


if __name__ == '__main__':
    main()
```



PyQt5

- 窗口
- 控件（Widget）
- 事件处理
- 布局管理

  - 事件：用户对界面上控件进行的操作
  - 事件源：事件发生的场所
  - 事件处理者：事件处理程序（方法或函数）

信号与槽用于对象之间的通信，是Qt库的核心机制

e.g. 在用户单击按钮的过程中，按钮单击事件是信号，槽是事件处理者（回调方法或函数）

PyQt5

事件

- 鼠标事件：`mousePressEvent / mouseReleaseEvent / mouseMoveEvent`
- 键盘事件：`keyPressEvent / keyReleaseEvent`

布局管理

- 控件的位置和大小不会随着容器的变化而变化
- 在不同平台上显示的效果可能差别很大
- 在不同分辨率下显示的效果可能差别很大
- 字体的变化也会对显示的效果有影响
- 在动态添加控件或删除控件后，界面需要重新布局

- 盒子布局管理器QBoxLayout
- 网格布局管理器QGridLayout
- 表单布局管理器QFormLayout

PyQt5

常见控件

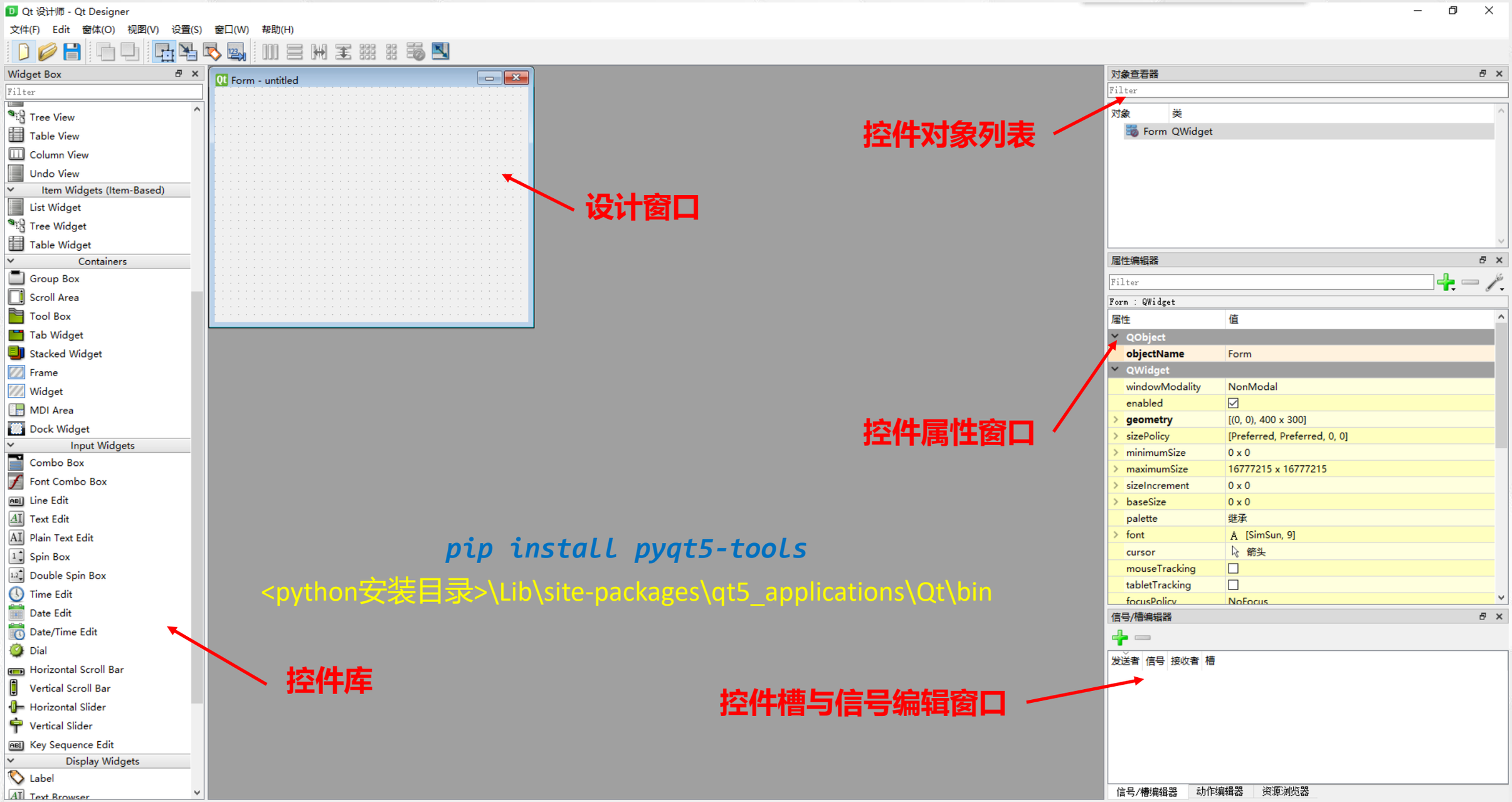
- 文本输入控件：单行输入控件QLineEdit / 多行输入控件QTextEdit
- 单选按钮：QRadioButton
- 复选框：QCheckBox
- 列表控件：QListWidget
- 下拉列表控件：QComboBox
- 表格控件：QTableWidget

PyQt5

PyQt5设计界面步骤



QTDesigner


Qt 设计师 - Qt Designer
文件(F) Edit 窗体(O) 视图(V) 设置(S) 窗口(W) 帮助(H)
Widget Box
Filter
Tree View
Table View
Column View
Undo View
Item Widgets (Item-Based)
List Widget
Tree Widget
Table Widget
Containers
Group Box
Scroll Area
Tool Box
Tab Widget
Stacked Widget
Frame
Widget
MDI Area
Dock Widget
Input Widgets
Combo Box
Font Combo Box
Line Edit
Text Edit
Plain Text Edit
Spin Box
Double Spin Box
Time Edit
Date Edit
Date/Time Edit
Dial
Horizontal Scroll Bar
Vertical Scroll Bar
Horizontal Slider
Vertical Slider
Key Sequence Edit
Display Widgets
Label
Text Browser
Form - untitled
对象查看器
Filter
对象
类
Form QWidget
属性编辑器
Filter
Form : QWidget
属性
值
QObject
objectName
Form
QWidget
windowModality
NonModal
enabled
geometry
[(0, 0), 400 x 300]
sizePolicy
[Preferred, Preferred, 0, 0]
minimumSize
0 x 0
maximumSize
16777215 x 16777215
sizeIncrement
0 x 0
baseSize
0 x 0
palette
继承
font
[SimSun, 9]
cursor
箭头
mouseTracking
tabletTracking
focusPolicy
NoFocus
信号/槽编辑器
发送者
信号
接收者
槽
信号/槽编辑器
动作编辑器
资源浏览器
控件对象列表
设计窗口
控件属性窗口
pip install pyqt5-tools
<python安装目录>\Lib\site-packages\qt5_applications\Qt\bin
控件库
控件槽与信号编辑窗口

Python库之图形用户界面

wxPython: 跨平台GUI开发框架

- 提供了专用于Python的跨平台GUI开发框架
- 理解数据类型与索引的关系，操作索引即操作数据
- Python最主要的数据分析功能库，基于Numpy开发

从人机交互到艺术设计

- Python库之图形用户界面
- Python库之游戏开发
- Python库之虚拟现实
- Python库之图形艺术

Python库之游戏开发

PyGame: 简单的游戏开发功能库

- 提供了基于SDL的简单游戏开发功能及实现引擎
- 理解游戏对外部输入的响应机制及角色构建和交互机制
- Python游戏入门最主要的第三方库

Python库之游戏开发

Panda3D: 开源、跨平台的3D渲染和游戏开发库





PANDA3D

http://www.panda3d.org

Python库之游戏开发

cocos2d: 构建2D游戏和图形界面交互式应用的框架

- 提供了基于OpenGL的游戏开发图形渲染功能
- 支持GPU加速，采用树形结构分层管理游戏对象类型
- 适用于2D专业级游戏开发

从人机交互到艺术设计

- Python库之图形用户界面
- Python库之游戏开发
- Python库之虚拟现实
- Python库之图形艺术

Python库之虚拟现实

VR Zero: 在树莓派上开发VR应用的Python库

- 提供大量与VR开发相关的功能
- 针对树莓派的VR开发库，支持设备小型化，配置简单化
- 非常适合初学者实践VR开发及应用

Python库之虚拟现实

pyovr: Oculus Rift的Python开发接口

- 针对Oculus VR设备的Python开发库
- 基于成熟的VR设备，提供全套文档，工业级应用设备
- Python+虚拟现实领域探索的一种思路

Python库之虚拟现实

Vizard: 基于Python的通用VR开发引擎

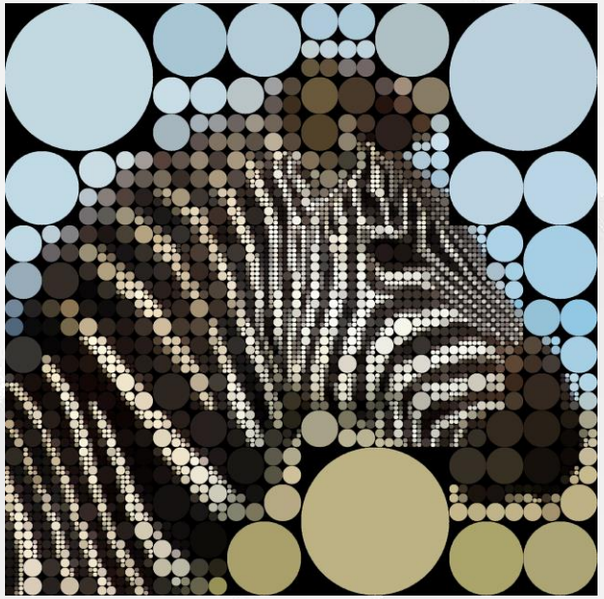
- 专业的企业级虚拟现实开发引擎
- 提供详细的官方文档
- 支持多种主流的VR硬件设备，具有一定通用性

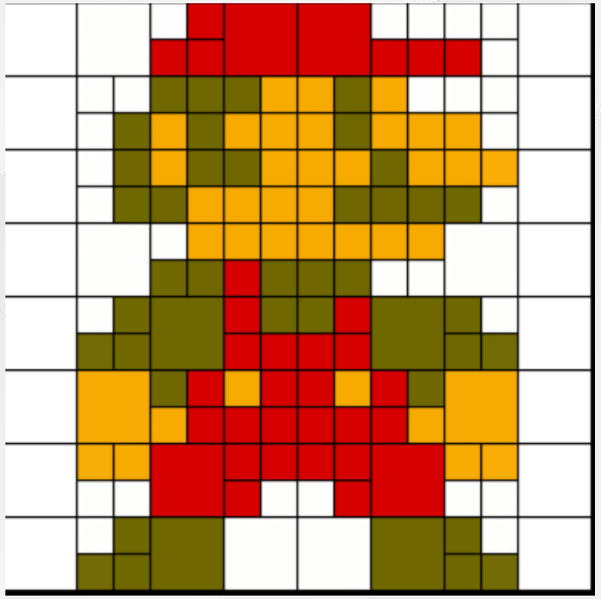
从人机交互到艺术设计

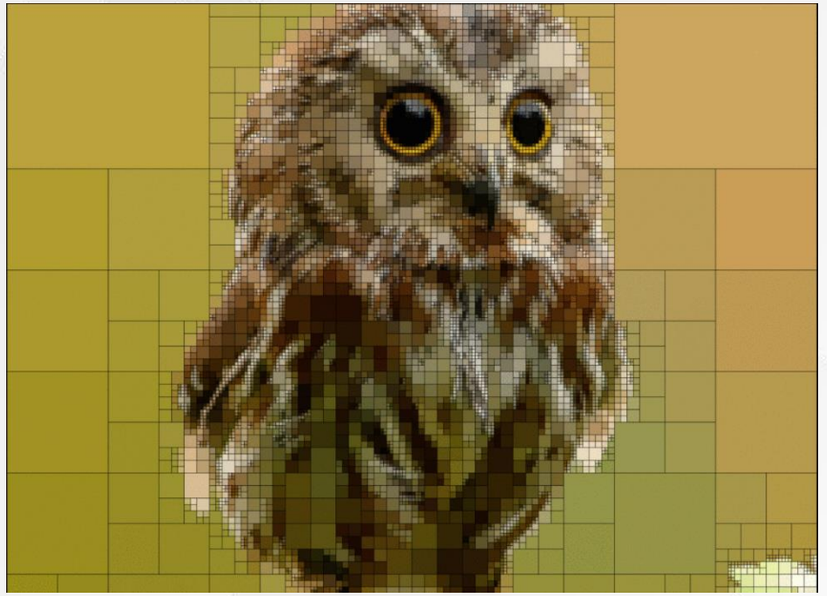
- Python库之图形用户界面
- Python库之游戏开发
- Python库之虚拟现实
- Python库之图形艺术

Python库之图形艺术

Quads: 迭代的艺术





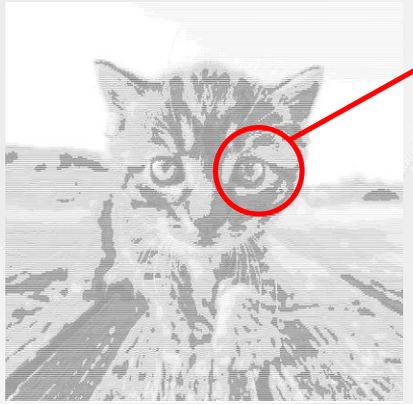


https://github.com/fogleman/Quads

Python库之图形艺术

ascii_art: ASCII艺术库





黑白



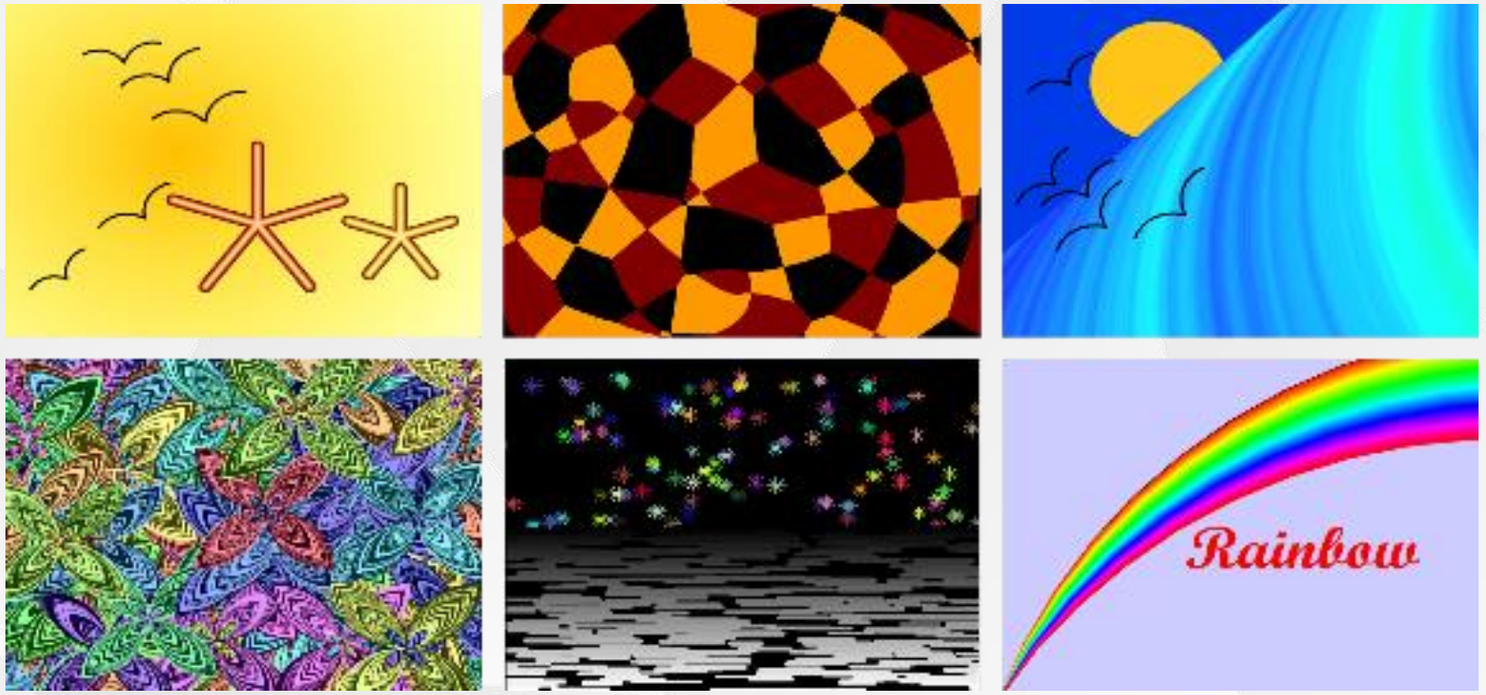
彩色

https://github.com/jontonsoup4/ascii_art

Python库之图形艺术

turtle: 海龟绘图体系

- Python语言的标准库之一
- 入门级的图形绘制函数库



https://docs.python.org/3/library/turtle.html

史骏

QQ: 55449053

Email: 55449053@qq.com

个人简历

包含：个人照片、自我介绍、学业成绩、项目经历、个人特长等



史骏
juns@hfut.edu.cn